

Audit technique et optimisation —

Activité 8

Objectif de l'activité : auditer le projet, identifier au moins deux axes d'optimisation, les prioriser et documenter les gains.

1. État initial audité

Axe	Constat
API	Routes REST nombreuses et fonctionnelles ; erreurs HTTP explicites ; healthcheck disponible
Frontend	Interface SPA fonctionnelle ; découpage en composants ; feedback utilisateur en cas d'erreur
Flux temps réel	SSE utilisé pour streamer la narration IA au lieu d'attendre une réponse complète
Sécurité	JWT + bcrypt ajoutés ; routes sensibles protégées
BDD	SQLite relationnel pour comptes et événements ; migration légère automatique
Déploiement	Docker Compose ; port dédié 8081 ; healthcheck backend

2. Problèmes détectés

Problème	Impact	Priorité
Ancienne identification par X-User-Id non sécurisée	Usurpation possible d'un joueur	P0
Routes de jeu accessibles sans token	Risque d'accès non autorisé	P0

Problème	Impact	Priorité
Message de fallback IA ambigu en cas d'erreur OpenAI billing	Confusion utilisateur / correcteur	P1
Certaines fonctions complexes (<code>combat_action</code> , parsing SSE)	Dette de maintenabilité	P2
CORS ouvert en développement	À restreindre en production	P2

3. Optimisations / corrections menées

Optimisation 1 — Sécurisation JWT complète

Avant : le frontend envoyait un simple en-tête `X-User-Id`, facile à falsifier.

Après :

- inscription et connexion via `/api/auth/register` et `/api/auth/login` ;
- génération de JWT signé ;
- envoi automatique `Authorization: Bearer <token>` côté frontend ;
- hachage bcrypt des mots de passe ;
- rôles `player` et `admin` ;
- routes sensibles protégées.

Preuves :

- [backend/auth.py](#)
- [backend/api.py](#)
- [frontend/src/api.js](#)
- [frontend/src/components/AuthPanel.jsx](#)

Gain : passage d'une identification déclarative à une authentification sécurisée et testable.

Optimisation 2 — Robustesse IA / fallback clarifié

Avant : en cas d'erreur OpenAI, le message pouvait indiquer à tort qu'aucune clé n'était détectée.

Après :

- distinction entre clé absente et OpenAI indisponible ;
- fallback local maintenu pour garantir une démonstration fonctionnelle ;
- healthcheck et jeu restent disponibles même si l'API OpenAI échoue.

Preuve : [backend/api.py](#), fonction `_gm_stream` et `_offline_gm_text`.

Gain : meilleure compréhension utilisateur, démo plus fiable.

Optimisation 3 — Validation automatique

Tests exécutés après modifications :

```
pytest -q
# 21 passed

cd frontend
npm test
# 13 passed

npm run build
# build OK
```

Gain : réduction du risque de régression avant rendu.

4. Mesures et validation

Validation	Résultat
Tests backend	21 passed
Tests frontend unitaires	13 passed
Build frontend	OK
Déploiement VPS	OK
Healthcheck prod	200 OK
Route protégée sans token	401

Validation	Résultat
Route protégée avec token	200

5. Optimisations restantes post-module

Axe	Action proposée
CORS	Remplacer <code>allow_origins=["*"]</code> par la liste des domaines autorisés
Refactoring	Découper <code>combat_action</code> et le parser SSE en fonctions plus petites
Base de données	Migrer les sauvegardes YAML critiques vers des tables SQL dédiées
Auth	Ajouter refresh token / rotation des tokens

6. Synthèse des gains (avant / après)

Critère	Avant	Après	Gain
Authentification	En-tête <code>X-User-Id</code> falsifiable	JWT signé HS256 + bcrypt	Usurpation d'identité éliminée
Accès aux routes de jeu	Ouvert sans contrôle	401 sans jeton valide	Surface d'attaque réduite
Stockage mot de passe	Potentiellement en clair	Hash bcrypt <code>\$2b\$...</code>	Confidentialité des identifiants
Robustesse IA	Message trompeur si OpenAI KO	Distinction clé absente / service KO + repli	Démo fiable, message clair
Perception latence narration	Attente réponse complète	Streaming SSE token par token	Retour visuel quasi immédiat
Couverture de non-régression	Aucune garantie automatisée	21 tests back + 13 front	Détection précoce des régressions

7. Perception performance — streaming SSE

Le passage d'une réponse « bloc complet » à un **flux SSE** ne réduit pas le temps de calcul total de l'IA, mais améliore fortement la **performance perçue** : l'utilisateur voit les premiers mots de la narration dès leur génération, au lieu d'attendre plusieurs secondes un écran figé. C'est une optimisation d'**expérience utilisateur** mesurable en « temps avant premier octet visible » (TTFB narratif).

8. Méthode d'audit appliquée

1. **Inventaire** des couches et des routes (17 endpoints, 11 modules domaine).
2. **Détection** des points faibles (sécurité, message d'erreur, complexité).
3. **Priorisation** P0/P1/P2 selon l'impact sécurité puis maintenabilité.
4. **Correction** des P0/P1 (sécurisation JWT, clarté fallback).
5. **Validation** par tests automatisés et vérifications en production.
6. **Documentation** des optimisations restantes pour la suite.